# CQRS Pattern: A Solution

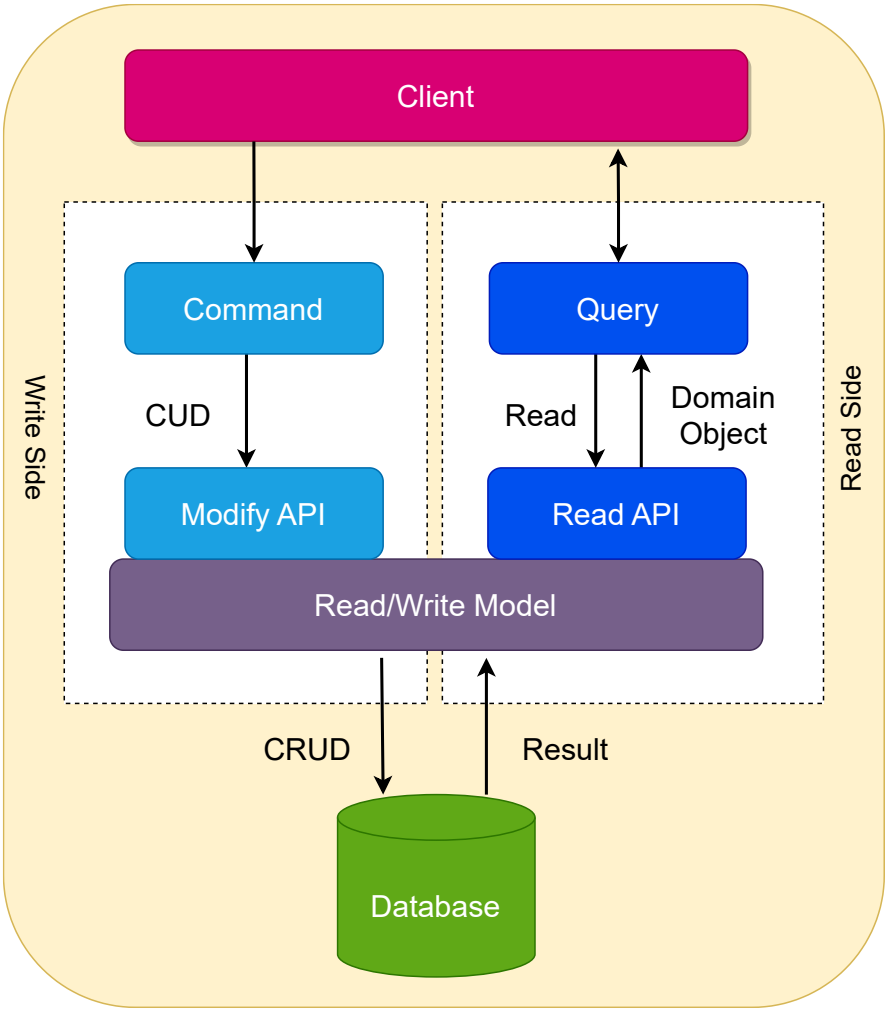
Command Query Responsibility Segregation (CQRS) is a design pattern in software architecture that helps solve the kind of problem we just discussed. To understand it better, let's break down the term:

- **Command:** This refers to an instruction to do something, like creating, updating, or deleting data (i.e., CUD operations). In our banking example, a command would be processing a deposit or a withdrawal.
- **Query:** This is a request to retrieve data without changing it. Checking your account balance in the banking app is a query.
- **Responsibility Segregation:** This means dividing responsibilities. In CQRS, it's about separating the responsibilities of handling commands (writes) and queries (reads).

Now, how does CQRS work in simple terms?

- **Separation of Concerns:** CQRS divides the system into two parts:
  - **The Command Side:** This part handles all the write operations. It's responsible for updating data, like processing transactions in our banking app.
  - **The Query Side:** This part deals with all the read operations. It's optimized for fetching data quickly, like showing your current account balance.

Here is how CQRS looks like with a single Read/Write model with single database.




Separating comamnds (writes) and queries (reads)

- **State Change**: The separation of concerns also distinguishes between methods that change the state and those that don't. This implies that each method of an object falls into one of these categories, but not both:
    - **Query:** Returns a result. Doesn't change the system's state or cause any side effects that change the state.
    - **Command** (also known as modifiers or mutators): Changes the state of a system. Doesn't return a value, only an indication of success or failure.
- **Why is this Useful**?
    - **Optimization:** By separating reads and writes, you can optimize each operation independently. The command side can focus on security and accuracy, while the query side can be optimized for speed.

- **Simplicity**: It simplifies the design. Each side (command and query) can be developed and maintained separately, reducing complexity.
- **Scalability**: As your application grows, you can scale the read and write operations independently, based on their specific demands.

In essence, CQRS acknowledges that the needs for reading and writing data are different and allows you to treat them separately. This leads to more efficient, maintainable, and scalable systems.

---

← Previous

The Problem: Traditional CRUD Operations

Next →

The Architecture Of The CQRS Pattern

Mark as Completed